

Intermediate (two-level logger) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/logging_practice.py` . Run it with `cd practice && python logging_practice.py` .

Used later by: the [Real-world wiring exercise](#) copies the `get_logger()` you write here into `practice/config_logging_wiring/logging_setup.py` , and the [Build Task](#) writes the same function again in `practice/build_task/logging_setup.py` . A copy, not an import — each of those folders stands alone. Keep the name and signature exactly as they are: `get_logger(name: str) -> logging.Logger` .

Story — `logging_practice.py` : `print()` gives you no way to separate "quiet, user-facing output" from "detailed diagnostic noise," and no record once the terminal scrolls away. `get_logger(name)` fixes both at once — quiet on screen, everything in a file. Written here, once, as its own exercise, because every later copy of this function (`config_logging_wiring`, `Build Task`, and beyond) builds on getting it right the first time. **If not:** the first time you'd wire up logging for real would be inside a bigger, more distracting file, with no smaller version to check your understanding against.

Basic Version

Approach 1 — the direct way

```
# practice/logging_practice.py
import logging

logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)

screen_handler = logging.StreamHandler()
screen_handler.setLevel(logging.INFO)
logger.addHandler(screen_handler)

file_handler = logging.FileHandler("app.log")
file_handler.setLevel(logging.DEBUG)
logger.addHandler(file_handler)

logger.debug("This only goes to the file.")
```

```
logger.info("This goes to both the screen and the file.")  
logger.warning("This also goes to both.")
```

Expected output (printed to your terminal):

```
This goes to both the screen and the file.  
This also goes to both.
```

The `debug` line is missing from the terminal on purpose — the screen handler's level is `INFO`, so it filters `DEBUG` out. It still reaches the file, because the file handler's level is `DEBUG`.

`app.log` after this runs (no formatter yet, so just the bare messages):

```
This only goes to the file.  
This goes to both the screen and the file.  
This also goes to both.
```

This version works correctly. It's missing type hints and a reusable function around the setup — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — wrapped in a typed function

```
# practice/logging_practice.py
import logging

def get_logger(name: str) -> logging.Logger:
    # why: one function every file can call to get a logger that prints
    # INFO+ to the screen and DEBUG+ to a file.
    logger = logging.getLogger(name)
    # how: the logger's own level must be the lowest of any handler, or messages
    # get filtered here first
    logger.setLevel(logging.DEBUG)

    screen_handler = logging.StreamHandler()
    # when: keeps the terminal quiet – only INFO and above show up live
    screen_handler.setLevel(logging.INFO)
    logger.addHandler(screen_handler)

    file_handler = logging.FileHandler("app.log")
    # when: the file keeps everything, including DEBUG, for reading after
    # something breaks
    file_handler.setLevel(logging.DEBUG)
    logger.addHandler(file_handler)

    return logger

logger = get_logger(__name__)

logger.debug("This only goes to the file.")
logger.info("This goes to both the screen and the file.")
logger.warning("This also goes to both.")
```

Expected output (terminal):

```
This goes to both the screen and the file.
```

This also goes to both.

app.log after this runs:

This only goes to the file.
 This goes to both the screen and the file.
 This also goes to both.

Approach 2 — same result, built from a list of handler specs

```
# practice/logging_practice.py
import logging

def get_logger(name: str) -> logging.Logger:
    """Create a logger that prints INFO+ to the screen and DEBUG+ to a file."""
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)

    handler_specs = [
        (logging.StreamHandler(), logging.INFO),
        (logging.FileHandler("app.log"), logging.DEBUG),
    ]
    for handler, level in handler_specs:
        handler.setLevel(level)
        logger.addHandler(handler)

    return logger

logger = get_logger(__name__)

logger.debug("This only goes to the file.")
logger.info("This goes to both the screen and the file.")
logger.warning("This also goes to both.")
```

Expected output (terminal):

This goes to both the screen and the file.
 This also goes to both.

app.log after this runs:

```
This only goes to the file.  
This goes to both the screen and the file.  
This also goes to both.
```

Difference from Basic: both Intermediate approaches move the setup into a typed function, `get_logger(name: str) -> logging.Logger`, so it can be reused for more than one logger without copy-pasting the whole block. Approach 2 also adds a docstring and loops over a list of `(handler, level)` pairs instead of writing out `.setLevel()` / `.addHandler()` twice by hand — purely a style choice for avoiding repetition, not a behavior difference from Approach 1.

Which one should you actually write? Intermediate Approach 1 (a typed `get_logger` function, no extra machinery) is completely correct and what most people should default to — it's the version the Build Task builds on next, adding the duplicate-handler guard and a formatter there.